Laya on the Snapdragon NPU: base vs fine-tuned

Incident-report classifier · Qualcomm SM8750P dev kit (Snapdragon 8 Elite, Hexagon v79 HTP) · QAIRT/SNPE 2.47 · 11 Oct 2026

Status: base model complete; fine-tuned model pending (training on Colab).

1. What is being compared

	Base Laya	Fine-tuned Laya
Weights	`convaiinnovations/laya` as published (revision `7b928d8`)	same encoder + LoRA adapters merged in + 3 new heads
Trained on our data?	**no**	yes: 1,164 training conversations (2,442 turn samples train, 275 validation)
How it predicts the incident type	zero-shot: cosine similarity between the user's text and one description per label written from `incident_taxonomy.json`	type head (30 classes)
Sentence relevance / form fields	not available (no heads)	relevance head + field head (60 fields)
Status	**done**	pending

Laya is a ModernBERT-large encoder (28 layers, hidden 1024, ~395M parameters) that its authors fine-tuned with reinforcement learning to decide between answering and escalating. Only the encoder is used here; its own decision heads are dropped.

Test set: the 128 held-out test conversations = **259 user-turn samples** (one per user turn, user text only, ≤ 128 tokens). Reported: type accuracy over all turns, on the final turn of each conversation, and macro-F1 over the 30 labels (28 types + `undetermined` + `none`). Chance is 1/30 = 3.3%.

2. Pipeline

1. **Export** (`laya/export.py`): PyTorch → ONNX, opset 17, static `[1, 128]` int32 inputs, eager attention. onnxruntime matches PyTorch to within 1.8e-7. 1.5 GB ONNX, 2,511 nodes, 7.5 s.
2. **Convert** (`qairt-converter`): ONNX → fp32 DLC, 55 s.
3. **Quantize** (`snpe-dlc-quant`, `laya/quant_variant.sh`): 50 calibration inputs drawn from the training set, 44–84 s.
4. **Prepare** (`snpe-dlc-graph-prepare --htp_socs sm8750`): HTP graph compiled offline, 8 s. First-load time on the device drops to ~0.1 s.
5. **Run** (`laya/run_device.sh`): `snpe-net-run --use_dsp`, burst mode, no CPU fallback. 289 inputs (259 test turns + 30 label descriptions for zero-shot) in 3–5 s.
6. **Score** (`laya/score.py`): the same metric code (`laya/common.py`) used on Colab and on the laptop.

3. Results: base Laya (no fine-tuning)

Base Laya	Agreement with fp32 (embedding cosine)	Type acc, all turns	Type acc, final turn	Macro-F1	NPU ms / turn	Init ms	File MB
fp32, laptop (reference)	1.000	**11.6%**	11.7%	0.077	—	—	1,507

Base Laya	Agreement with fp32 (embedding cosine)	Type acc, all turns	Type acc, final turn	Macro-F1	NPU ms / turn	Init ms	File MB
W8A16 on NPU (8-bit weights, 16-bit activations)	**0.985**	**9.3%**	7.8%	0.057	**10.8**	107	859
W8A8 on NPU (all 8-bit)	0.126	4.2%	5.5%	0.028	6.8	94	759

File size includes the HTP graph prepared for SM8750. The 8-bit weights alone are 378 MB.

Reading it:

- **Un-fine-tuned Laya barely does the task.** Even at full precision it is right 11.6% of the time, against 3.3% chance. Its embeddings are tuned for answer/escalate decisions, not for matching a report to an incident-type description, and they are very similar for every input (mean cosine 0.90 between any two test turns).
- **W8A16 is a faithful NPU copy.** Its embeddings agree with full precision (mean cosine 0.985). Only 3 of 289 inputs drift, all 7–9 tokens long ("hi" style openings). Accuracy drops from 11.6% to 9.3% at 10.8 ms per turn.
- **Fully 8-bit (W8A8) does not work for Laya.** Its output is unrelated to full precision (cosine 0.13), and 3 inputs come back as all-zero vectors. Accuracy is at chance.

3b. Flat vs two-stage (department first), and unquantized Laya on the device

Two-stage: step 1 picks the department (Facilities, Housekeeping, Security, EHS, IT, Labs) or `undetermined` / `none`, 8 options; step 2 picks the incident type only among that department's types. Both steps use the same embedding from one forward pass on one loaded model. Base Laya produces one embedding per text, so a second pass over the same text would give identical numbers; only the set of descriptions compared against changes. Department descriptions come only from the taxonomy (department name, technician roles, its incident labels).

Unquantized on the device: this SDK runs only quantized models on the Hexagon NPU, so the fp32 model was run on the dev kit's CPU and GPU.

Base Laya	Runs on	Flat (30 labels)	Two-stage	Stage-1 dept acc	ms / turn	Init ms
fp32, laptop	PyTorch	11.6%	8.9%	10.0%	—	—
fp32, dev kit	Kryo CPU	**11.6%**	8.9%	10.0%	495	506
fp32, dev kit	Adreno GPU (fp32 math)	12.4%	8.9%	10.0%	80	3,266
W8A16, dev kit	Hexagon NPU	9.3%	8.5%	8.5%	**10.8**	107
W8A8, dev kit	Hexagon NPU	4.2%	4.2%	13.1%	6.8	87

Chance: 3.3% for 30 labels, 12.5% for the 8 stage-1 options.

- **The device runs the model correctly.** The fp32 run on the kit's CPU reproduces the laptop exactly (same numbers to the decimal); the GPU is within a point.
- **Two-stage does not help base Laya; it hurts (8.9% vs 11.6%).** Step 1 picks the right department only 10% of the time, at or below chance, so step 2 usually searches the wrong department. Base Laya's embedding does not carry department-level information either, so narrowing the choice cannot fix it.
- **Speed:** the NPU (W8A16) is 46× faster than the device CPU and 7× faster than the GPU, for about 2 points of accuracy.

4. Quantization findings (what broke and how it was fixed)

Problem 1: the attention mask (fixed). ModernBERT masks padding by adding `float32.min` (−3.4e38) to the attention scores. In a quantized graph that one value sets the range of the whole attention-score tensor, and every

real score collapses to the same level, so every input gets nearly the same embedding. Before the fix, all variants failed:

Before the fix (original mask)	Cosine to fp32	Type acc	NPU ms
W8A8 (default min-max)	0.176	3.9%	10.1
W8A8 + per-channel + percentile calibration + 32-bit bias	0.275	3.9%	10.2
W8A16 + per-channel + 32-bit bias	0.140	6.2%	18.8

Fix (`export_friendly_masks` in `laya/common.py`): build the same masks with **−60** instead of −3.4e38. After softmax a −60 entry still gets weight < 1e-26, and full-precision output is bit-identical (max difference 0.0). It applies only at export time; training is unaffected. With it, W8A16 went from cosine 0.14 to 0.985, and its latency also fell from 18.8 to 10.8 ms.

Problem 2: 8-bit activations (not fixable by calibration). Even with the mask fixed, per-channel weights and 32-bit biases, W8A8 Laya is unusable. Large encoders have a few very large activation values that one 8-bit range per tensor cannot hold together with the small ones. The workable NPU format for Laya is **8-bit weights + 16-bit activations**, which the v79 HTP runs natively.

5. Results: fine-tuned Laya

Pending. The model is training on Colab (`laya/laya_finetune.ipynb`). It will be exported with the same fixed masks, run as fp32 on the kit's CPU, W8A16 and W8A8 on the NPU, on the same 259 test turns, and added here, scored both flat and two-stage (department probabilities summed from the type head, then the best type within the chosen department).

Fine-tuned Laya	Type acc, all turns	Final turn	Macro-F1	Relevance acc	Fields micro-F1	NPU ms	File MB
fp32 (Colab GPU)	—	—	—	—	—	—	—
W8A16 on NPU	—	—	—	—	—	—	—
W8A8 on NPU	—	—	—	—	—	—	—

6. Limits

- **Small test set:** 259 turns from 128 conversations; ±3–4 points is within noise.
- **Synthetic data:** the test set is LLM-written like the training data, so absolute numbers will be optimistic for the fine-tuned model. A team-written test set is still needed.
- **The zero-shot baseline depends on the label descriptions,** which were built only from the taxonomy (label plus English seed phrases). Better descriptions might raise it a little; they would not make it competitive.
- **Timing covers only the model on the NPU,** not tokenization or app overhead.

7. Reproduce

```
source qairt-env.sh
python laya/export.py base                          # ONNX + calibration/test inputs
qairt-converter --input_network laya/out/base/model.onnx --output_path laya/out/base/model.dlc
laya/quant_variant.sh base w8a16 --act_bitwidth 16 --bias_bitwidth 32 --
use_per_channel_quantization
```

```
laya/quant_variant.sh base w8a8  --use_per_channel_quantization --bias_bitwidth 32
ANDROID_SERIAL=fe7980db laya/run_device.sh base w8a16 && python laya/score.py base w8a16
# fine-tuned: python laya/export.py ft laya/laya_ft.pt, then the same with "ft"
```